Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра И5 «Информационные системы и программная инженерия»

Практическая работа №1
по дисциплине «Информатика: Основы программирования»
на тему «Структура программы, основные типы данных, ввод/вывод»

Выполнил:
Студент Праудин Д.С.
Группа 0717b

Преподаватель:
Палехова О.А.

Санкт-Петербург
2020 г.

Задание 1.

Написать программу, которая будет находить сумму любых двух целых чисел, введенных с клавиатуры.

Входные данные: `слагаемые, два целых числа. Обозначим их a и b`, тип int.
Выходные данные: `сумма, целое число. Обозначим как s`, тип int.

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
a = 2, b = 2	4	4
a = 2000, b = -2000	0	0
a = 2000000000, b = 2000000000	4000000000	-

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b, s;          /* объявление переменных */
    printf ("a = ");      /* печать сообщения */
    scanf ("%d", &a);     /* ввод с клавиатуры вещественного числа и запись его в переменную a */
    printf ("b = ");      /* печать следующего сообщения */
    scanf ("%d", &b);     /* ввод с клавиатуры вещественного числа и запись его в переменную b */
    res = a + b;          /* вычисление суммы и запись ее в переменную res */
    printf ("%d + %d = %d\n", a, b, s); /* вывод результата в формате число + число = число */
    return 0;
}
```

Выводы: `При вводе слишком большого числа происходит переполнение`

Задание 2.

Написать программу деления одного целого числа на другое.

Входные данные:`делимое,делитель,два целых числа. Обозначим их a и b, тип int.`

Выходные данные: `частное,целое число. Обозначим как res, тип int.`

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
a=18 b=3	6	6
a=15 b= 2	7,5	7
a=20 b=30	0,6666666667	0
a=20 b=0	-	-

Текст программы:

```
#include <stdio.h>

#include <stdlib.h>


int main()

{
```

```
    int a, b, res;
    printf ("a = ");
    scanf ("%d", &a);
    printf ("b = ");
    scanf ("%d", &b);
    res = a / b;
    printf ("%d / %d = %d\n", a, b, res);
    return 0;
}
```

Выводы: программа успешно вычисляет целое значение деления одного целого числа на другое целое число. При делении на ноль происходит ошибка.

Задание 3.

Изменить тип переменных в предыдущей программе на *double* (стандартный вещественный тип). В функциях *scanf()* и *printf()* поменять спецификаторы формата на *%lf*. *Входные* и *выходные данные* те же, что и в задании 2, обозначения переменных те же, тип всех переменных double.

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
a=18, b=3	6	6
a=15, b= 2	7,5	7
a=20, b=30	0,67	0.666667
a=20, b=0	-	-
a=4.2, b=2.1	2	2.000000
a=6.8, b=3.2	2.125	2.125000
a=5.5, b=0.1	55	55.000000

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    double a, b, res;
    printf ("a = ");
    scanf ("%lf", &a);
    printf ("b = ");
    scanf ("%lf", &b);
    res = a / b;
    printf ("%lf / %lf = %lf\n", a, b, res);
    return 0;
```

```
}
```

Выводы: программа успешно делит как целые числа, так и дробные. Выводимый на экран результат деления округляется по математическим правилам.

При изменении формата вывода на *%.4lf* выводимое значение стало таким: *0.6667*.

При изменении формата вывода на *%.2lf* выводимое значение стало таким: *0.67*.

Выводы:при выводе в консоль чисел, используя форматирование, можно вручную менять количество цифр после запятой.

Задание 4.

В предыдущей программе изменить тип делимого и делителя обратно на int, результат оставить типа double.

Входные данные : делимое число и делитель. Обозначим их a и b, тип int.

Выходные данные: результат деления, целое число. Обозначим как res, тип double.

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
a=18, b=3	6	6
a=15, b= 2	7,5	7
a=20, b=30	0,67	0.666667
a=20, b=0	-	-
a=4.2, b=2.1	2	Неверно считываются данные вещественных чисел.
a=6.8, b=3.2	2.125	Неверно считываются данные вещественных чисел.
a=5.5, b=0.1	55	Неверно считываются данные вещественных чисел.

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
int main()

{
    int a, b;
    double res;
    printf ("a = ");
    scanf ("%d", &a);
    printf ("b = ");
    scanf ("%d", &b);
    res = (double) a / b;
    printf ("%d / %d = %lf\n", a, b, res);
```

```
    return 0;
}
```

Выводы: при делении int на int получается int, при делении double на int получается double.

Задание 5.

Проанализировать ошибки при вызове функций scanf() и printf().

Ошибка	Поведение программы
отсутствие & перед именем переменной в scanf()	Программа аварийно завершается, процесс возвращает значение -1073741819 (0xC0000005).
наличие & перед именем переменной в printf() при выводе значения переменной	Неверно считывается переменная
тип спецификатора формата ввода не совпадает с типом переменной: переменная типа *int*, спецификатор *%lf*	Программа неверно считывает переменную, вместо заданного значения подставляется 0.
тип спецификатора формата ввода не совпадает с типом переменной: переменная типа *double*, спецификатор *%d*	Программа вместо переменной подставляет 0.
тип спецификатора формата ввода не совпадает с типом переменной: переменная типа *double*, спецификатор *%f*	Программа записывает переменную некорректно.
тип спецификатора формата вывода не совпадает с типом значения: значение типа *int*, спецификатор *%lf*	Программа вместо переменной подставляет 0.
тип спецификатора формата вывода не совпадает с типом значения: значение типа *double*, спецификатор *%d*	Программа вместо переменной подставляет 0.
количество спецификаторов формата ввода меньше количества вводимых значений переменных	В переменной остается значение которое было в переменной при объявлении.
количество спецификаторов формата ввода больше количества вводимых значений переменных	Программа аварийно завершается,процесс возвращения значения -1073741819(0xc0000005).
количество спецификаторов формата вывода меньше количества выводимых значений	Выводится значения только тех спецификаторов на которые хватило значения.

количество спецификаторов формата вывода больше количества выводимых значений	При каждом запуске программа с одинаковыми числами лишний спецификатор выдает разные числа.

Задание 6.

Познакомиться с типами данных *char* и *unsigned char*.

```
#include <stdlib.h>
#include <stdio.h>
#include <limits.h>     /*подключение заголовочного файла с константами
максимальных и минимальных значений для различных типов переменных */
int main()
{
    char c;
    unsigned char uc;
    printf("sizeof(c)=%d\tsizeof(uc)=%d\n\n", sizeof(c),sizeof(uc)); /* вывод в
консоль  переменных char и unsigned char  */
    uc = c = CHAR_MAX;  /* приравнивание переменных uc и c к максимальному значению
переменной типа char; обе переменные примут значение 127 */
    printf("CHAR_MAX : c=%d uc=%d\n", c, uc); /* вывод в консоль значений
переменных c и uc */
  c = c + 1; uc = uc + 1;     /* прибавление единицы к переменным c и uc
соответственно; в итоге переменная c примет значение -128 (числа больше 127 в
переменную этого типа записать нельзя), а uc будет равно 128 */
    printf("CHAR_MAX+1 : c=%d uc=%d\n", c, uc);
    uc = c = CHAR_MIN;  /* присваивание переменных uc и c к минимальному значению
переменной типа char; c=-128, а uc=128 (т.к. в uc невозможно записать
отрицательные числа, знак минуса игнорируется) */
    printf("CHAR_MIN : c=%d uc=%d\n", c, uc);
    c = uc = UCHAR_MAX; /* присваивание переменных c и uc к максимальному значению
переменной типа unsigned char; в переменную c это значение записывается
некорректно, так как оно больше 127 */
    printf("UCHAR_MAX : c=%d uc=%d\n", c, uc);
    c = c + 1; uc = uc + 1;   /* прибавление единицы к переменным c и uc
соответственно; uc=0 т.к. старший 9й разряд при записи суммы игнорируется */
    printf("UCHAR_MAX+1 : c=%d uc=%d\n", c, uc);
    uc = c = -5;  /* приравнивание c и uc к -5; c=-5, а uc=251, так как uc не
считает старший разряд за наличие знака минус (имеет беззнаковый тип переменной)
*/
  printf("-5 : c=%d uc=%d\n", c, uc);
    c = -5; uc = 5;     /* присваивание отрицательного значения для переменной c и
положительного значения для переменной uc */
    printf("char and unsigned char -5>5 : %d\n\n", c>uc);  /* вывод в консоль
утверждения: является ли c больше uc; выполняется корректное сравнение, т.к. обе
переменные имеют тип char */
    return 0;
}
```

Результаты работы программы:

```
sizeof(c)=1     sizeof(uc)=1

CHAR_MAX : c=127 uc=127
CHAR_MAX+1 : c=-128 uc=128
CHAR_MIN : c=-128 uc=128
UCHAR_MAX : c=-1 uc=255
UCHAR_MAX+1 : c=0 uc=0
-5 : c=-5 uc=251
char and unsigned char -5>5 : 0


Process returned 0 (0x0)   execution time : 0.048 s
Press any key to continue.
```

Задание 7.

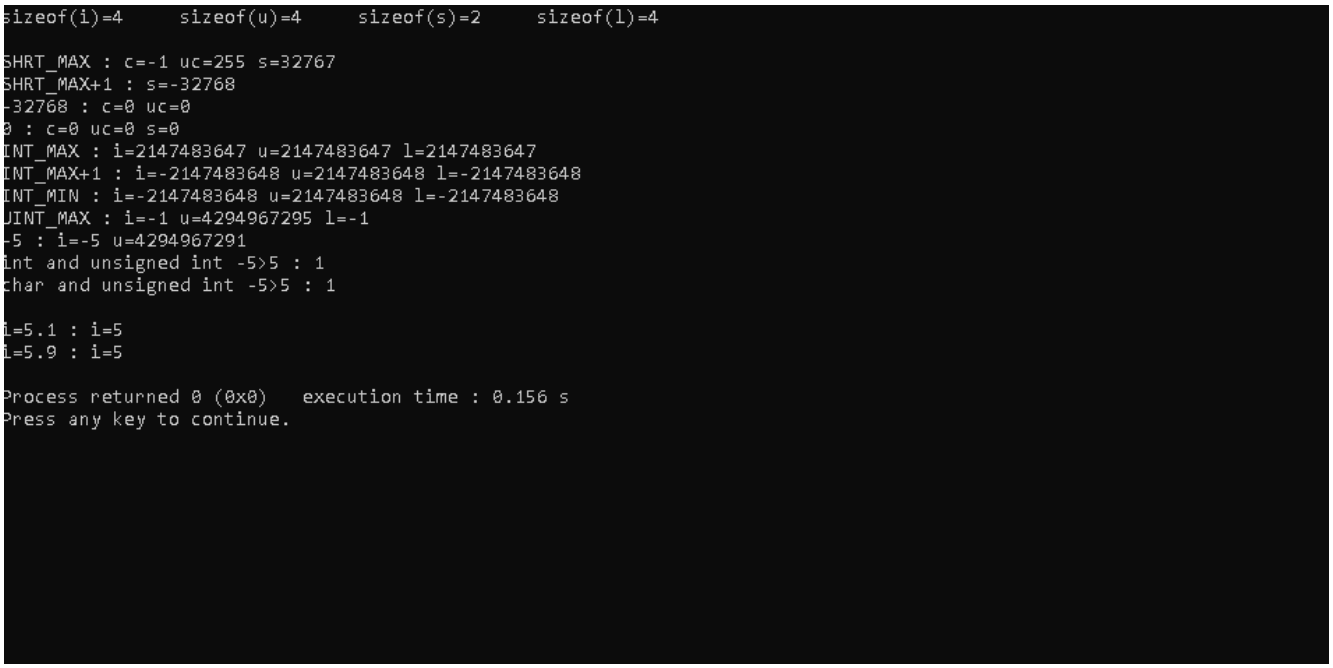
Познакомиться с типами данных *int*, *short int*, *long int* и *unsigned int*.

```
#include <stdlib.h>
#include <stdio.h>
#include <limits.h>

int main()
{
    char c;
    unsigned char uc;
    int i;
    unsigned u;
    short s;
    long l;
    printf("sizeof(i)=%d\tsizeof(u)=%d\tsizeof(s)=%d\tsizeof(l)=%d\n\n",
          sizeof(i), sizeof(u), sizeof(s), sizeof(l));     /* вывод длины
переменных i,u,s,l в байтах */
    c = s = SHRT_MAX;   /* присваивание переменных c и s к максимальному значению
переменной типа short */
   uc = s;        /* присваивание переменной uc к значению переменной s, т.е. к
максимальному значению переменной типа short */
    printf("SHRT_MAX : c=%d uc=%d s=%d\n", c, uc, s);
    s = s + 1;    /* т.к. 32767+1 > 32767, то по принципу цикличности значений
переменная s будет равна -32768 */
   printf("SHRT_MAX+1 : s=%d\n", s);
    c = s; uc = s;      /* присваивание переменной c к -32768; в обоих переменных
будет записан 0 (т.к. более старшие разряды невозможно записать в переменные типа
char и unsigned char) */
 printf("%d : c=%d uc=%d\n", SHRT_MIN, c, uc);
    s = 0; c = s; uc = s;     /* присваиваем переменные s,c,uc к нулю */
    printf("0 : c=%d uc=%d s=%d\n", c, uc, s);
    i = INT_MAX;  /* присваиваем переменную i к её максимуму */
  l = i; u = i;   /* присваиваем переменные l и u к максимуму переменной типа int
*/
    printf("INT_MAX : i=%d u=%u l=%ld\n", i, u, l);  /* значения были записаны и
выводятся корректно */
   i = i + 1; l = l + 1; u = u + 1; /* прибавляем единицу к каждой переменной
i,l,u (по отдельности); переменная l из-за цикличности примет отрицательное
значение */
    printf("INT_MAX+1 : i=%d u=%u l=%ld\n", i, u, l);
    i = INT_MIN; /* присваиваем переменной i её минимум */
    l = i; u = i; /* присваиваем переменные l и u к минимуму int; в переменной u
знака минуса нет, т.к. в эту переменную можно записывать неотрицательные значения
*/
  printf("INT_MIN : i=%d u=%u l=%ld\n", i, u, l);
    u = UINT_MAX; /* присваиваем переменной u её максимум */
```

```
    i = u; l = u; /* присваиваем переменным i и l минимум переменной типа unsigned;
в i и l значение будет записано некорректно т.к. оно выходит за рамки допустимых
значений */
 printf("UINT_MAX : i=%d u=%u l=%ld\n", i, u, l);
    u = i = -5; /* присваиваем отрицательные значения к переменным u и i; u будет
равно 4294967291 в силу цикличности */
    printf("-5 : i=%d u=%u\n", i, u);
    i = -5; u = 5;      /* присваивание переменным i и u соответствующие значение -
5 и 5; сравнение i > u будет выполнено некорректно, так как переменные имеют
разные типы */
  printf("int and unsigned int -5>5 : %d\n", i > u);
    c = -5; u = 5;      /* присваивание переменным c и u соответствующие значение -
5 и 5; сравнение c > u будет выполнено некорректно, так как переменные имеют
разные типы */
  printf("char and unsigned int -5>5 : %d\n\n", c > u);
    i = 5.1;      /* дробная часть не записывается в переменную, вместо 5.1 будет
5 */
  printf("i=5.1 : i=%d\n", i);
    i = 5.9;      /* дробная часть не записывается в переменную, вместо 5.9 будет
5 */
  printf("i=5.9 : i=%d\n", i);
    return 0;
}
```

Результаты работы программы:



```
sizeof(i)=4     sizeof(u)=4     sizeof(s)=2     sizeof(l)=4

SHRT_MAX : c=-1 uc=255 s=32767
SHRT_MAX+1 : s=-32768
-32768 : c=0 uc=0
0 : c=0 uc=0 s=0
INT_MAX : i=2147483647 u=2147483647 l=2147483647
INT_MAX+1 : i=-2147483648 u=2147483648 l=-2147483648
INT_MIN : i=-2147483648 u=2147483648 l=-2147483648
UINT_MAX : i=-1 u=4294967295 l=-1
-5 : i=-5 u=4294967291
int and unsigned int -5>5 : 1
char and unsigned int -5>5 : 1

i=5.1 : i=5
i=5.9 : i=5

Process returned 0 (0x0)   execution time : 0.156 s
Press any key to continue.
```

Задание 8.

Познакомиться с типами данных *float* и *double*.

```
#include <stdlib.h>
#include <stdio.h>
#include <float.h>      /* подключение заголовочного файла с константами различных
ограничений и параметров типов с плавающей точкой */

int main()
{
    float f;
    double d;
```

```
    printf("sizeof(f)=%d\tsizeof(d)=%d\n\n", sizeof(f), sizeof(d)); /* вычисляем
длину в байтах переменных float и double соответственно */
 d = f = FLT_MAX; /* приравниваем переменные к максимальному значению типа float */
printf("FLT_MAX : f=%g d=%g\n", f, d);
    d = f = FLT_MIN;    /* приравниваем переменные к минимальному значению типа
float */
   printf("FLT_MIN : f=%g d=%g\n", f, d);
    d = f = FLT_EPSILON;      /* присваивание переменным значение машинного
эпсилона (минимальное близкое к нулю дробное значение, которое можно присвоить
переменной вещественного типа; минимальный "шаг" переменной типа float) */
printf("FLT_EPSILON : f=%g d=%g\n", f, d);
    f = 1e10;     /* присваивание переменной десятичного числа 1 в десятой степени
*/
 printf("1e10 : f=%f\n", f);
    f = 1e11;     /* присваивание переменной десятичного числа 1 в 11 степени;
число будет записано некорректно, т.к. значение выйдет за границы максимума */
printf("1e11 : f=%f\n", f);
    f = 1234567890;     /* из-за большого количества разрядов в float точность
изначального числа теряется */    printf("1234567890 : f=%f\n", f);
    d = DBL_MAX;  /* присвоение максимума типа double */
    printf("DBL_MAX : d=%g\n", d);
    d = DBL_MIN;  /* присвоение минимума типа double */
    printf("DBL_MIN : d=%g\n", d);
    d = DBL_EPSILON;    /* присвоение машинного эпсилона для типа double */
printf("DBL_EPSILON : d=%g\n", d);
    d = 1e15 + 1; /* присвоение числа 1 в степени 15 плюс 1 */
  printf("1e15+1 : d=%lf\n", d);
    d = 1e16 + 1; /* присвоение числа 1 в степени 16 плюс 1; точность числа
теряется */
    printf("1e16+1 : d=%lf\n", d);
    f = 10000 * 100000;
    f += 1;
    f -= 4 * 250000000; /* если поочередно применить операции сложения и вычитания
больших чисел, то в переменной типа float итоговое значение будет искажено (вместо
единицы мы получим ноль) */
  printf("1 : f=%f\n", f);
    f = 10000 * 100000 + 1 - 4 * 250000000; /* если выполнить все операции в одну
строку и записать итоговый результат в переменную float, то мы получим верное
значение */
  printf("1 : f=%f\n", f);
    d = 10000 * 100000 + 1 - 4 * 250000000;/* аналогично с переменной типа double,
в данном случае мы получим единицу */
  printf("1 : d=%lf\n", d);
    d = 1e20 * 1e20 + 1000 - 1e22 * 1e18; /* при вычислительных операциях с
большими числами теряется точность итогового значения; в данном случае вместо 1000
мы получим 0 */
    printf("1000 : d=%lf\n", d);
    d = 1e20 * 1e20 - 1e22 * 1e18 + 1000; /* если прибавить 1000 в конце, а не в
середине выражения, то мы получим верный результат *
    printf("1000 : d=%lf\n", d);
    return 0;
}
```

Результаты работы программы:

```
sizeof(f)=4     sizeof(d)=8

FLT_MAX : f=3.40282e+038 d=3.40282e+038
FLT_MIN : f=1.17549e-038 d=1.17549e-038
FLT_EPSILON : f=1.19209e-007 d=1.19209e-007
1e10 : f=10000000000.000000
1e11 : f=99999997952.000000
1234567890 : f=1234567936.000000
DBL_MAX : d=1.79769e+308
DBL_MIN : d=2.22507e-308
DBL_EPSILON : d=2.22045e-016
1e15+1 : d=1000000000000001.000000
1e16+1 : d=10000000000000000.000000
1 : f=0.000000
1 : f=1.000000
1 : d=1.000000
1000 : d=0.000000
1000 : d=1000.000000

Process returned 0 (0x0)   execution time : 0.222 s
Press any key to continue.
```

Задание 9.

Проверить порядок выполнения операций в каждом выражении, содержащем несколько операций присваивания, разделив каждый оператор-выражение на несколько операторов, выполняемых последовательно.

Текст измененной программы:

```
#include <stdlib.h>

int main ()
{
    int a, b = 5, c;
    double x, y = -.5, z;
    printf("a=");
    scanf("%d", &a);
    c=a;
    x=c;
    printf("x = c = a : a=%d c=%d x=%f\n", a, c, x);
    a += b;
    printf("a += b : a=%d\n", a);
    x *= b + a;
    printf("x *= b+a : x=%lf\n", x);
    b += a;
    a -= 1;
    printf("b += a-- : a=%d b=%d\n", a, b);
    c += 1;
    x -= c;
    printf("x -= ++c : c=%d x=%lf\n", c, x);
    c = a / b;
    printf("c = a/b : c=%4d\n",c);
    c = a % b;
    printf("c = a%%b : c=%d\n",c);
    y += (a + 1) / a;
    a += 1;
    printf("y += (a+1)/a++ : a=%d y=%.3lf\ty=%.0lf\n", a, y, y);
```

```
    y -= .6;
    b = 3 * y + 2 * b + 1;
    printf("b = 3*(y-=.6)+2*b+1 : b=%d y=%.1lf\n", b, y);
    z = a / 2;
    printf("z = a/2 : z = a/2 : z=%lf\n", z);
    z = (double)a / 2;
    printf("z = (double)a/2 : z=%lf\n", z);
    x = 5.7;
    y = x / 2;
    printf("y = (x = 5.7)/2 : x=%lf y=%lf\n", x, y);
    y = (int)x / 2;
    printf("y = (int)x/2 : y=%f\n", y);
    c = c / 2;
    z = ( b - 3 ) / 2 - x / 5 + c + 1 / 4 * z - y;
    b += 1;
    y += 1;
    z += b / 3.;
    printf("z = (b-3)/2 - x/5 +(c/=2) + 1/4*z - y++ + ++b/3. :\n\
    a=%d b=%d c=%d x=%lf y=%lf z=%lf\n", a, b, c, x, y, z);
    return 0;
}
```

}Результаты работы программ:

до изменения

```
a=12
x = c = a : a=12 c=12 x=12.000000
a += b : a=17
x *= b+a : x=264.000000
b += a-- : a=16 b=22
x -= ++c : c=13 x=251.000000
c = a/b : c=   0
c = a%b : c=16
y += (a+1)/a++ : a=17 y=0.500   y=1
b = 3*(y-=.6)+2*b+1 : b=44 y=-0.1
z = a/2 : z = a/2 : z=8.000000
z = (double)a/2 : z=8.500000
y = (x = 5.7)/2 : x=5.700000 y=2.850000
y = (int)x/2 : y=2.000000
z = (b-3)/2 - x/5 +(c/=2) + 1/4*z - y++ + ++b/3. :
    a=17 b=45 c=8 x=5.700000 y=3.000000 z=39.860000

Process returned 0 (0x0)   execution time : 2.642 s
Press any key to continue.
```

после изменения

```
a=12
x = c = a : a=12 c=12 x=12.000000
a += b : a=17
x *= b+a : x=264.000000
b += a-- : a=16 b=22
x -= ++c : c=13 x=251.000000
c = a/b : c=   0
c = a%b : c=16
y += (a+1)/a++ : a=17 y=0.500   y=1
b = 3*(y-=.6)+2*b+1 : b=44 y=-0.1
z = a/2 : z = a/2 : z=8.000000
z = (double)a/2 : z=8.500000
y = (x = 5.7)/2 : x=5.700000 y=2.850000
y = (int)x/2 : y=2.000000
z = (b-3)/2 - x/5 +(c/=2) + 1/4*z - y++ + ++b/3. :
    a=17 b=45 c=8 x=5.700000 y=3.000000 z=39.860000

Process returned 0 (0x0)   execution time : 2.057 s
Press any key to continue.
```

Задание 10.

Написать программу для вычисления значений следующих выражений:

a=5, c=5

a=a+b-2

c=c+1, d=c-a+d

a=a*c, c=c-1

a=a/10, c=c/2, b=b-1, d=d*(c+b+a)

Выражения, записанные в одной строке, записывать одним оператором-выражением, не содержащим запятой. Использовать расширенные операции присваивания, операции инкремента и декремента. Переменные c и d объявить как целые, переменные a и b – как вещественные. Значения переменных b и d вводить с клавиатуры. После вычисления каждого выражения выводить на экран значения всех переменных.

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
10,2	a=7.800000 b=9.000000 c=2 d=-94	a=7.800000 b=9.000000 c=2 d=-94
1.3,50	a=2.580000 b=0.300000 c=2 d=248	a=2.580000 b=0.300000 c=2 d=248
13,0.8	a=9.600000 b=12.000000 c=2 d=-236	a=9.600000 b=12.000000 c=2 d=-236

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int c,d;
    double a,b;

    printf("b=");
    scanf("%lf", &b);

    printf("\nd=");
    scanf("%d", &d);


    a = c = 5;
    a += b - 2;
    d += ++c - a;
    a *= c--;
    d *= ((c /= 2)+(--b)+(a /= 10));


    printf("\n\na=%lf\n", a);
    printf("b=%lf\n", b);
    printf("c=%d\n", c);
    printf("d=%d\n", d);

    return 0;
}
```